MANIPAL UNIVERSITY
JAIPUR

PROJECT REPORT

AceOS

Asset, Compliance & Equipment
Operations System

Relational Database Management Systems

Submitted by

Smit Hitnalli

Registration No: 2502051938
Section: N3

Manipal University Jaipur
B.Tech Computer Science and Engineering

Abstract and report guide

AceOS provides a traceable workflow for institutional equipment operations. Its repair process separates technician completion from administrator verification and retains every correction as a new submission. Complaint reasons, repair reviews, audit events and safety evidence make the outcome explainable. Relational constraints and guarded service operations work together to preserve consistency. The implementation also demonstrates encapsulated lifecycle methods, JavaFX inheritance, JDBC interface polymorphism, generics and transaction resource management. The result is a working local college project whose documented verification includes 89 configured checks with no failures, errors or skips in the final recorded run. This report uses implementation evidence rather than claiming new experiments were performed during report preparation.

Report structure

This submission focuses on Relational Database Management Systems. The following page guide includes the cover and references within the required 20 pages.

Page	Topic
3	Project background and objectives
4	Requirements and application scope
5	Database architecture
6	Entity relationships: repair workflow
7	Entity relationships: safety and accounts
8	Data dictionary: identities and assets
9	Data dictionary: complaints and work
10	Data dictionary: submissions and reviews
11	Data dictionary: inspection and safety
12	Data dictionary: schedules and infrastructure
13	Normalization and deliberate redundancy
14	Keys, constraints and integrity
15	SQL queries and JDBC access
16	ACID and the repair transaction
17	Concurrency, stale screens and history
18	Database demonstration and verification
19	Limitations, future scope and conclusion
20	References and source evidence

Project background and objectives

AceOS stands for Asset, Compliance & Equipment Operations System. It is a local JavaFX application for managing institutional assets, complaints, repairs, inspections and preventive schedules. The same implemented application supports two academic perspectives: relational data management and object-oriented software construction.

Campus equipment maintenance involves several linked facts: which asset has a problem, who reported it, which technician was assigned, what work was submitted and which administrator accepted the result. A collection of disconnected spreadsheets makes these links difficult to validate and can lose the history of a correction.

AceOS connects those records through defined identities and workflows. Requesters report faults and track their own complaints. Technicians execute assigned work. Administrators review complaints, coordinate assignments and verify completed repairs. Inspection failures can also restrict an asset independently of its repair history.

Implemented scope

The report documents the current v2.0.0 application and schema migrations V001-V005. It does not describe a proposed online campus portal. The application runs locally using Java 24, JavaFX 24.0.2, JDBC, MySQL 8 and Maven. A separately configured database and private credentials are required.

Requirements and application scope

The database must support an asset register, role identities, authenticated accounts and traceable maintenance. Every complaint needs an asset and reporter. Accepted complaints may create a repair assigned to a technician. The database must distinguish an unstarted assignment, work in progress, completion awaiting review and final acceptance.

Role	Responsibility	Key restriction
Requester	Report, comment, withdraw, reopen	Own complaints; lifecycle rules apply
Technician	Repair, correct and inspect	Assigned work and permitted history
Administrator	Review, assign, verify, manage	Required reasons and evidence checks

Data requirements

History must remain linked to its original records. Withdrawal and reopening preserve the complaint identifier and original reporting time. Corrections retain earlier submissions rather than overwriting the work description. Account deactivation must preserve user references already used by historical operations.

Consistency requirements

A failed inspection creates its own safety hold. Repair verification resolves only the linked complaint and must not clear unrelated holds or overlapping repairs. Related state, evidence and audit writes either commit together or roll back together. Inputs are bound through prepared statements.

Storage boundary

MySQL stores durable state. JavaFX displays authorized projections, and services coordinate policy and transactions. The Notion documentation hub is a companion knowledge source; it is not an application database.

Database architecture

The application follows a view-service-repository-database path. Views request actions from AceOsService. The service validates the session and role, coordinates a transaction and calls repositories. Repositories issue SQL using JDBC and reconstruct models or record projections.

Layer	Responsibility
JavaFX views	Collect input; present authorized results
AceOsService	Authorize and coordinate one business action
Transactions	Own commit, rollback and connection lifetime
Repositories	Bind parameters; query and persist records
MySQL	Enforce keys, value constraints and durability

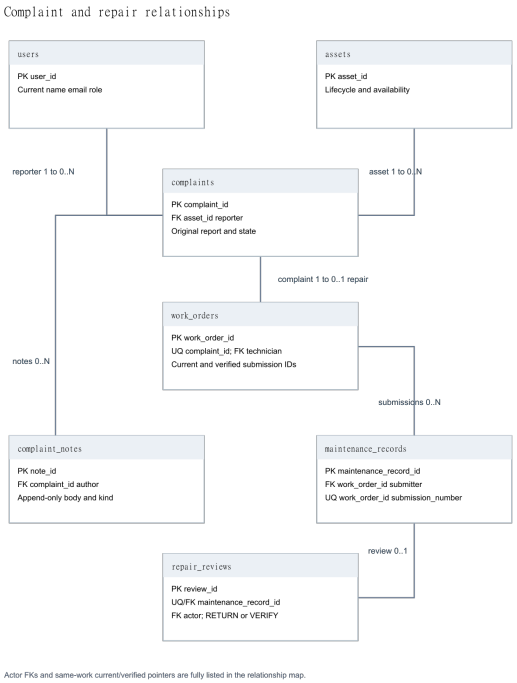
Transaction ownership

Repositories borrow a transaction-bound Connection. The proxy suppresses repository commit, close and setAutoCommit calls so a nested repository cannot accidentally finish the outer service transaction. Only the transaction owner commits and closes the underlying connection.

Schema evolution

A fresh database uses V000 baseline followed by matching COMPLETE migrations V001-V005. Version/checksum metadata is checked at runtime. Migration is an explicit operator activity rather than an automatic sign-in action. DDL migration recovery is separate from rollback-safe application transactions.

Entity relationships: repair workflow

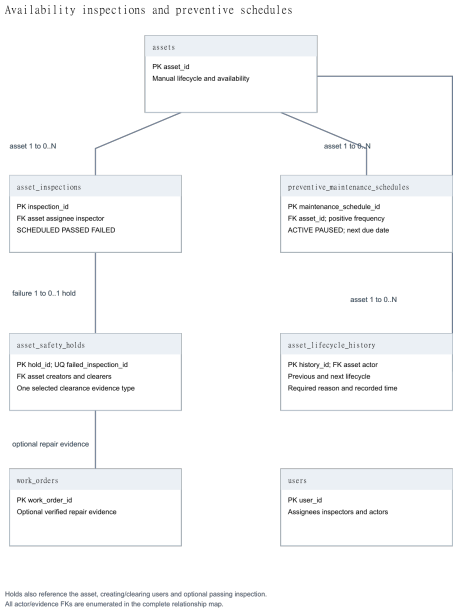


Repair and complaint relationships; source: docs/diagrams/aceos-er-repairs.svg.

An asset can have many complaints, each reported by one user. A complaint has zero or one work order, enforced by a unique complaint_id in work_orders. One repair may have several maintenance_records, and each submission may have at most one terminal administrator review.

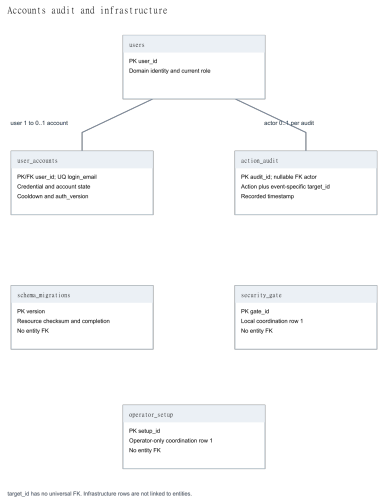
The current and verified submission pointers are optional composite foreign keys. They include work_order_id so a pointer cannot reference another repair's evidence. Actor relationships are summarized in the printable diagram; the source relationship map enumerates all foreign keys.

Entity relationships: safety and accounts



Availability, inspection and preventive-schedule relationships.

Inspection failures create individual holds. Clearance records an administrator, reason, time and exactly one selected evidence type. The service additionally validates that the evidence concerns the same asset and occurs at an eligible later time.



Accounts and infrastructure tables; metadata tables have no invented relationships.

Data dictionary: identities and assets

users

Column	Type	Null?
user_id	int	NO
name	varchar(100)	NO
email	varchar(255)	NO
role	varchar(20)	NO

user_accounts

Column	Type	Null?
user_id	int	NO
login_email	varchar(255)	NO
password_hash	varchar(255)	YES
state	varchar(25)	NO
must_change	tinyint(1)	NO
temporary_expires_at	timestamp	YES
failed_attempts	int	NO
failure_window	timestamp	YES
locked_until	timestamp	YES
auth_version	bigint	NO

assets

Column	Type	Null?
asset_id	int	NO
asset_code	varchar(50)	NO
name	varchar(100)	NO
category	varchar(100)	NO
location	varchar(150)	NO
status	varchar(30)	NO
manual_lifecycle	varchar(30)	NO

users.email and user_accounts.login_email are separately unique. user_accounts.user_id is both primary key and foreign key to users. Authentication state is separated from domain identity so existing operational references remain intact.

Data dictionary: complaints and work

complaints

Column	Type	Null?
complaint_id	int	NO
asset_id	int	NO
reported_by_user_id	int	NO
description	text	NO
priority	varchar(20)	NO
status	varchar(30)	NO
reported_at	datetime	NO

work_orders

Column	Type	Null?
work_order_id	int	NO
complaint_id	int	NO
technician_user_id	int	NO
status	varchar(20)	NO
assigned_at	datetime	NO
started_at	datetime	YES
completed_at	datetime	YES
verified_at	datetime(6)	YES
current_submission_id	int	YES
verified_submission_id	int	YES

complaint_notes

Column	Type	Null?
note_id	bigint	NO
complaint_id	int	NO
author_user_id	int	NO
author_role	varchar(20)	NO
kind	varchar(20)	NO
body	varchar(2000)	NO
recorded_at	timestamp	NO

work_orders.complaint_id is unique. Current and verified submission links require evidence from the same work order. Notes are append-only at runtime and preserve required review, withdrawal and reopening reasons.

Data dictionary: submissions and reviews

maintenance_records

Column	Type	Null?
maintenance_record_id	int	NO
work_order_id	int	NO
action_taken	text	NO
recorded_at	datetime(6)	NO
submission_number	int	NO
submitted_by_user_id	int	YES
author_name	varchar(100)	YES
author_role	varchar(20)	YES

repair_reviews

Column	Type	Null?
review_id	int	NO
maintenance_record_id	int	NO
kind	varchar(10)	NO
actor_user_id	int	NO
author_name	varchar(100)	NO
author_role	varchar(20)	NO
reason	varchar(2000)	YES
recorded_at	datetime(6)	NO

A unique pair of work_order_id and submission_number gives each correction an ordered identity. A unique maintenance_record_id in repair_reviews allows at most one terminal RETURN or VERIFY review per submission. Author name/role snapshots preserve the attribution recorded at posting.

Data dictionary: inspection and safety

asset_inspections

Column	Type	Null?
inspection_id	int	NO
asset_id	int	NO
inspection_type	varchar(150)	NO
scheduled_date	date	NO
status	varchar(20)	NO
inspected_by_user_id	int	YES
notes	text	YES
completed_date	date	YES
assigned_to_user_id	int	YES
completed_at	datetime(6)	YES

asset_safety_holds

Column	Type	Null?
hold_id	int	NO
asset_id	int	NO
failed_inspection_id	int	NO
created_by_user_id	int	NO
created_at	datetime(6)	NO
cleared_by_user_id	int	YES
clearance_reason	varchar(1000)	YES
passing_inspection_id	int	YES
verified_work_order_id	int	YES
cleared_at	datetime(6)	YES

Each failed_inspection_id is unique in asset_safety_holds. An active hold has no clearance fields. A cleared hold requires actor, reason, time and one evidence type. A later passing inspection alone does not clear an older hold.

Data dictionary: schedules and infrastructure

preventive_maintenance_schedules

Column	Type	Null?
maintenance_schedule_id	int	NO
asset_id	int	NO
task_description	varchar(255)	NO
frequency_days	int	NO
next_due_date	date	NO
status	varchar(20)	NO

asset_lifecycle_history

Column	Type	Null?
history_id	int	NO
asset_id	int	NO
actor_user_id	int	NO
previous_lifecycle	varchar(30)	NO
next_lifecycle	varchar(30)	NO
reason	varchar(1000)	NO
changed_at	datetime(6)	NO

action_audit

Column	Type	Null?
audit_id	bigint	NO
actor_user_id	int	YES
action	varchar(80)	NO
target_id	int	YES
recorded_at	timestamp	NO

Coordination and migration metadata

Table	Columns and meaning
security_gate	gate_id INT PK; singleton local coordination row
operator_setup	setup_id INT PK; operator-only coordination
schema_migrations	version INT PK; checksum CHAR(64); status VARCHAR(20)

All metadata columns above are NOT NULL. Runtime cannot access operator_setup and reads schema_migrations without updating it. Migration state is STARTED or COMPLETE; matching versions and checksums are required.

Normalization and deliberate redundancy

The relational decomposition separates identities, assets, complaints, work orders, submissions and reviews. Each table describes a distinct kind of fact rather than repeating an entire complaint and asset record for every repair submission. This reduces update anomalies and makes history independently addressable.

Functional dependencies

Determinant	Dependent attributes / meaning
asset_id	Code, name, category, location and lifecycle
complaint_id	Asset, reporter, description, priority, state
work_order_id	Complaint, technician and progress timestamps
work_order_id + submission_number	One numbered repair submission
maintenance_record_id in reviews	At most one terminal review

Normal-form reasoning

Fields store scalar values, and repeated submissions are separate rows rather than repeating columns. Non-key facts are organized around their table identity. This is a practical normalization analysis, not a formal proof that every table is in BCNF under all possible business dependencies.

Intentional stored values

assets.status stores a recalculated availability result even though it depends on lifecycle, holds and repairs. Author-name and role snapshots repeat historical attribution intentionally. These choices trade strict removal of redundancy for stable historical meaning and convenient reads. Their consistency depends on transactionally coordinated writes.

Keys, constraints and integrity

Primary keys identify rows, foreign keys preserve references, unique keys express business cardinalities, and check constraints restrict valid values and evidence shape. These mechanisms reject malformed persistence even when an application operation contains a defect.

Rule	Implemented mechanism
One repair per complaint	UNIQUE(work_orders.complaint_id)
Numbered submissions	UNIQUE(work_order_id, submission_number)
One terminal review	UNIQUE(repair_reviews.maintenance_record_id)
Same-work evidence	Composite current/verified submission FKs
Valid role and state values	Constrained strings mapped to Java enums
Positive recurrence	frequency_days > 0
One hold per failed inspection	UNIQUE(failed_inspection_id)

Policy beyond keys

A foreign key proves that a referenced user exists; it does not prove that the user is currently a technician allowed to perform the action. Likewise, an evidence foreign key alone does not establish that a passing inspection occurred after a failure on the same asset. Services therefore validate roles, ownership, time and expected state.

action_audit.target_id intentionally has no generic foreign key because its target type depends on action. The report does not invent relationships for security_gate, operator_setup or schema_migrations.

SQL queries and JDBC access

The following illustrative read-only queries use real table and column names. They explain joins and aggregation; no result counts or performance measurements are claimed.

Complaint, asset and reporter join

```
SELECT c.complaint_id, a.asset_code, u.name, c.status
FROM complaints c
JOIN assets a ON a.asset_id = c.asset_id
JOIN users u ON u.user_id = c.reported_by_user_id
ORDER BY c.reported_at DESC;
```

Unverified workload per technician

```
SELECT u.user_id, u.name, COUNT(w.work_order_id) AS jobs
FROM users u LEFT JOIN work_orders w
  ON w.technician_user_id = u.user_id
 AND w.status <> 'VERIFIED'
WHERE u.role = 'TECHNICIAN'
GROUP BY u.user_id, u.name;
```

The LEFT JOIN retains technicians with zero matching jobs. COUNT(w.work_order_id) counts matched repairs rather than counting the outer-join row. Keeping the repair predicate in ON preserves those zero-workload users.

Parameterized access

```
SELECT maintenance_record_id, submission_number, action_taken
FROM maintenance_records WHERE work_order_id = ?
ORDER BY submission_number;
```

The question mark is a JDBC placeholder bound before execution. Application queries also apply authorization scope; the illustrative SQL alone is not the complete permission policy.

ACID and the repair transaction

Repair verification must accept one exact submission, close the linked complaint and update availability without leaving partial results. AceOsService uses one authorized transaction across those writes and the audit event.

Property	AceOS example
Atomicity	Review, state, complaint and audit commit together
Consistency	Keys and guarded expected-state updates remain valid
Isolation	READ COMMITTED plus gate and asset row locks
Durability	A successful database commit persists the accepted action

Verification sequence

```
BEGIN authorized administrator transaction
Validate session, role and schema
Lock coordination gate and relevant asset
Check COMPLETED + expected latest submission
Insert VERIFY review for that submission
Set VERIFIED and accepted submission pointer
Resolve linked complaint; recalculate availability
Append audit event; COMMIT
```

An exception before commit causes the owner to roll back. If audit insertion fails, an accepted review and resolved complaint must not survive alone. Another active repair or safety hold remains authoritative even after this complaint is resolved.

This is explanatory pseudocode for implemented behaviour, not a migration script or an instruction to reset the database.

Concurrency, stale screens and history

A stale screen can still show a repair as COMPLETED after another actor has returned it and a technician has submitted a correction. Checking status alone would allow the administrator to accept work that was never displayed. AceOS therefore compares the exact expected submission identifier.

Correction cycle

Step	Persistent effect
Administrator returns submission 1	RETURN review with reason; status RETURNED
Assigned technician corrects work	Insert submission 2; move back to COMPLETED
Old screen attempts verification	Expected submission mismatch rejects action
Fresh screen verifies submission 2	VERIFY review; exact accepted pointer retained

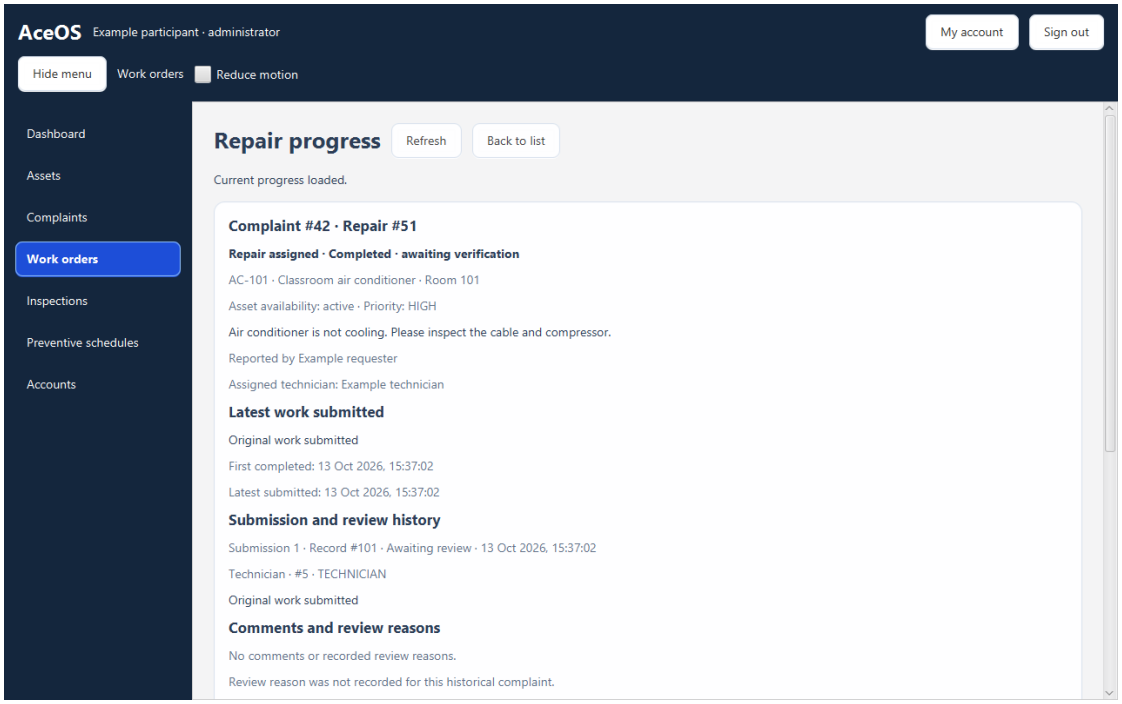
A correction retains assigned_at, started_at and first completed_at. The new record has its own submission time. Unique submission numbering and conditional updates prevent competing resubmissions from both succeeding for one return.

Coordination policy

A single security_gate row deliberately serializes service operations for the local college application. Asset locks coordinate availability and evidence changes. READ COMMITTED avoids an early repeatable-read snapshot hiding new work or holds while an operation waits for the gate. This is a correctness-oriented local design, not a scalability benchmark.

The next inspection date uses completion date plus frequency. Preventive completion neither clears a safety hold nor creates a preventive work-order history.

Database demonstration and verification



Controlled administrator repair-review fixture; source: target/ui-checks.

The recorded final v2.0.0 verification run passed 89 checks: 46 database checks, 3 password/session checks and 40 JavaFX checks, with zero failures, errors or skips. These figures come from the project verification documentation; the application test suite was not rerun while preparing this report.

Area	Recorded evidence
Relational behaviour	Rollback, stale evidence, competing actions, foreign keys
Authorization	Role, ownership and invalidated-session checks
Repair history	Return, correction, exact latest-submission verification
Interface	Draft retention, duplicate-save guards and recovery
Human walkthrough	Five repair-return steps confirmed by the project author

The focused repair-return checks overlap the final suite and must not be added to 89 as extra coverage. UI snapshots are controlled verification fixtures, not production usage statistics. Layout review was recorded at 800 x 520 and 1280 x 800.

Interpretation

The evidence supports the implemented workflows within the configured local environment. It does not establish production scalability, remote security enforcement or performance under large institutional workloads. Database tests require the disposable test schema; a skipped database test is not evidence of correctness.

Limitations, future scope and conclusion

AceOS meets the closed college scope for local asset and maintenance operations. It preserves repair history, prevents stale verification and keeps independent safety restrictions authoritative. The main design result is that complaint closure, repair acceptance and asset availability remain distinct decisions.

Current limitations

The trusted local Java client is the authorization boundary. A modified client or someone holding database credentials can bypass Java checks. The system does not provide a hosted backend, remote enforcement, cancellation of verified repairs, reassignment after repair start or automatic clearance of safety holds. Preventive completion advances the next due date without a separate completion-history table.

Future scope

Possible extensions include notifications, QR-based asset lookup, attachments, calendar integration and richer reporting. A future shared deployment would need server-side authorization and an explicit concurrency and operational design. These are proposed extensions, not features claimed in the submitted application.

Conclusion

The project demonstrates how clear responsibilities and explicit evidence links improve correctness. Its most instructive case is a returned repair: the technician adds a correction, the administrator accepts one exact submission, and the system preserves the earlier submissions and reviews for later explanation.

References and source evidence

Primary evidence is the AceOS repository and its existing documentation. Paths below are repository-relative. The report describes current v2.0.0 behaviour; the v2.0.0 release tag identifies the packaged source.

Reference	Purpose
README.md; docs/college-scope.md	Application scope and release context
docs/data-dictionary.md	All 16 tables, fields and key rules
docs/er-diagram.md	Complete foreign-key relationship map
docs/transactions.md	Atomic operations and consistency policy
docs/oop-design.md	Implemented OOP concept mapping
docs/verification.md	Recorded verification and limitations
docs/foundation.md; docs/run-guide.md	Setup, migration and recovery
pom.xml	Build dependencies and application version
src/main/resources/database/V000-V005	Baseline and versioned schema resources
src/main/java/com/aceos/	Application implementation
src/test/java/com/aceos/	Database, security and UI checks

Repository: https://github.com/SmitHitnalli/aceos

Evidence conventions

SQL examples marked illustrative explain the schema and are not measured query results. Java excerpts are drawn from named source files; simplified diagrams describe implemented relationships and are not claims of additional runtime classes. Screenshots are copied from the controlled UI-check evidence. No passwords, connection secrets or private database rows are included.